

Exposing signals to C++

Not every internal net is visible to your C++ testbench by default

Public, hidden, and ports

- Ports are your contract, no pragma required for normal top-level access patterns
- Unmarked internals are HIDDEN from the host's perspective unless you peek via hierarchy
- Mark selectively with the public comment when a scoreboard or checker in C++ must read an
- More exposure slows compile and couples TB to implementation details

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Verilator public

INTERACTIVE TOOL

Verilator public

Document-aid literacy: `/*verilator public*/` and hierarchy visibility for the C++ TB (not a real Verilator run). Starter: `u_core.state` marked public — `VISIBLE`.

Starter example: `top.u_core.state` with `/*verilator public*/` — `VISIBLE` to the C++ TB.

Load starter example

Challenges 0 / 22 cleared

Quiz: `public: /*verilator public*/` is used to...

☐ keep a signal visible to the C++ testbench hierarchy

☐ enable `--trace-fst` automatically

☐ set the timescale

☐ silence `UNUSED` warnings

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

public

Keep a signal visible to the C++ TB.

public_flat

Flatten name into the parent scope.

hierarchy

Deep internals need an explicit mark.

ports

Top ports are already TB-visible.

Lab

SCENARIO

Lab starter

learn_verilator — verilator public

Real Verilator practice

- In Track A, take a tiny module with one internal counter
- Run once without public markers and note what the host cannot see
- Add a public marker on the one net you must observe, recompile, and confirm access
- Remove gratuitous markers before you commit, minimal is the rule

Pitfalls to watch

- Do not mark every signal public to avoid thinking about interfaces
- Do not assume ports need the pragma, they already belong to the boundary
- Do not hide required check signals and then debug blind in C++
- And remember recompile is required after marker changes, this is not a runtime poke

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, pass minimal-marking challenges
- In Track A, expose one internal net intentionally and read it from the host
- When you are ready, take the short quiz, then continue to verification metrics